

Assignment Module 3: Quantum Neural Networks (QNN)

This assignment focuses on quantum neural networks (QNNs) for binary classification using a PennyLane + PyTorch implementation of a fully quantum classifier. To complete the assignment, you will implement missing code (marked **YOUR CODE HERE**) and answer a set of short theoretical questions.

Preparation

- Look at the notebooks and slides on **Quantum Neural Networks (QNNs)** and **PennyLane-PyTorch integration** provided in this module.

Use of generative AI tools

You may use AI-based tools (e.g., ChatGPT, GitHub Copilot, Claude, Gemini, DeepSeek, ...) for brainstorming, refactoring, coding assistance, plotting, or editing.

This is allowed with disclosure. LLMs are a great tool, but you have to make sure to grasp the contents of the course!

Make sure to fill in the mandatory AI-disclosure in the notebook before submitting!

Preparatory code

Run this cell to import the modules we need.

```
In [1]: # Reproducibility
SEED = 123

# Imports
import math, sys, os, json, pathlib
import numpy as np
np.random.seed(SEED)

import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
)

import pennylane as qml
from pennylane import numpy as pnp # optional

import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

print("pennylane version: ", qml.__version__)
print("torch version:      ", torch.__version__)

# no need to use GPU for this assignment
device_torch = "cuda" if torch.cuda.is_available() else "cpu"
print("Torch device:", device_torch)

print("Imports OK")

FP_TOL = 0.05 # floating point tolerance for automated tests
```

pennylane version: 0.42.3
torch version: 2.9.1+cpu
Torch device: cpu
Imports OK

Task 0: Loading the dataset

In this exercise, we will use the digits dataset from scikit-learn, but we restrict ourselves to a binary classification problem (two digits only, e.g., digits 0 and 1).

Before designing the QNN, we must first:

- Load the dataset and select only two classes (0 and 1).
- Split the data points into a training, validation and test sets.
- Apply a scaling to the input features using `StandardScaler()`, which ensures that the features have zero mean and unit variance.

Remember to fit the scaler on the training data, and use the same fitted scaler on both the training, validation and test datasets!

```
In [2]: # Data Loading and preprocessing
# TODO: Load digits dataset, pick two classes (0 and 1), train/test split, and scale features.

from sklearn.datasets import load_digits

# Load full digits dataset
digits = load_digits()

# All data and labels. Each x is the pixel value of the digit between 0 and 16
```

```
# in a 8 x 8 image, and each y is the associated target value
X_all = digits.data      # shape: (n_samples, n_features)
y_all = digits.target    # shape: (n_samples,)

# -----YOUR CODE HERE-----
# 1. Create a boolean mask that selects only digits 0 and 1.
mask = np.zeros_like(y_all, dtype=bool)
mask = (y_all == 0) | (y_all == 1)

# 2. Apply the mask to X_all and y_all to get a binary dataset.
X = X_all[mask][:]
y01 = y_all[mask]

# 3. Split the dataset in 70% training, 15% validation and 15% test.
# Remember to use the seed random_state=SEED, and to stratify according to y01 and y_temp.
X_tr, X_temp, y_tr, y_temp = train_test_split(X,
                                              y01,
                                              test_size=0.3,
                                              random_state=SEED,
                                              stratify=y01
                                              )
X_val, X_te, y_val, y_te = train_test_split(X_temp,
                                              y_temp,
                                              test_size=0.5,
                                              random_state=SEED,
                                              stratify=y_temp
                                              )

# print (X.shape)
# print (len(X))
# print (X_tr.shape)
# print (X_temp.shape)
# print (X_val.shape, X_te.shape)

# 4. Make mean 0 and standard deviation 1 using StandardScaler
scaler = StandardScaler()
X_tr = scaler.fit_transform(X=X_tr)
X_val = scaler.transform(X=X_val)
X_te = scaler.transform(X=X_te)
# ---YOUR CODE ENDS HERE---

# print (X_tr)

print("Shapes:", X_tr.shape, X_val.shape, X_te.shape,
      " (class 1 count):", (y_tr==1).sum(),
      " (class 0 count):", (y_tr==0).sum())

print("Test mean/std:", X_te.mean().round(6), X_te.std().round(6))

assert len(X_tr) + len(X_val) + len(X_te) == len(X)
assert set(np.unique(y_tr)).issubset({0,1})
assert X_tr.shape == (252, 64)
assert X_val.shape == (54, 64)
assert X_te.shape == (54, 64)
assert (y_tr==1).sum() == 127
assert (y_tr==0).sum() == 125
assert np.isclose(X_tr.mean(), 0.0, FP_TOL, FP_TOL)
assert np.isclose(X_te.mean(), -0.033565, FP_TOL, FP_TOL)
```

Shapes: (252, 64) (54, 64) (54, 64) (class 1 count): 127 (class 0 count): 125
Test mean/std: -0.027555 0.864232

Question 0.1 — Short answer

Why is feature scaling important when we use angle-embedding (e.g. RY rotations) to encode classical data into a QNN?
Write a concise justification (2–4 sentences).

Answer: Feature scaling is important when we use angle-embedding because angle-embedding directly converts each data point to a rotation angle. If the data is not normalized then angle wrapping can occur for data values that are larger than 2π . Angle wrapping occurs for all data points x that satisfy $x = x + 2\pi k$, where k is an integer, and maps unique inputs to the same quantum state.

Question 0.2 — Short answer

What does the parameter stratify do? (1-2 sentences).

Answer: The stratify parameter in the `sklearn.model_selection.train_test_split` function allows `train_test_split` to split a dataset into train and test sets while preserving the ratio of classes specified in the dataset that the keyword `stratify` is assigned to. Without using the stratify parameter the test and train datasets are not guaranteed to hold a percentage of each class that is equal to the dataset that was split.

Task 1: Dimensionality reduction using PCA

For angle-based maps we often set the number of qubits equal (or proportional) to the feature dimension. Unfortunately, the available qubit count is often smaller than the dimensionality of the data.

Here, we use PCA to reduce the data dimensionality to `n_qubits` components, so that we can encode the data on a small QNN.

Instructions:

1. Choose `n_qubits` (e.g., 4).
2. Fit `PCA(n_components=n_qubits)` on the training data and transform both training, validation and test sets.
3. Print shapes to confirm.

```
In [3]: # PCA reduction to match qubits
# -----YOUR CODE HERE-----
n_qubits = 4
pca = PCA(n_components=n_qubits)
Xtr_red = pca.fit_transform(X=X_tr)
Xval_red = pca.transform(X=X_val)
Xte_red = pca.transform(X=X_te)
# ---YOUR CODE ENDS HERE---
print("Reduced shapes:", Xtr_red.shape, Xval_red.shape, Xte_red.shape)
assert Xtr_red.shape == (252, 4)
assert Xval_red.shape == (54, 4)
assert Xte_red.shape == (54, 4)
```

Reduced shapes: (252, 4) (54, 4) (54, 4)

Question 1.1 — Short answer

Explain briefly (2–3 sentences) why PCA is useful in this QNN setting, and what we are trading off by compressing the features.

Answer: PCA is useful for this QNN because it allows us to train our QNN fewer qubits, which is expected to train faster and less computationally expensive. Additionally, we expect PCA to work well for the digit classification problem because there are many pixels that will not be used for the drawing of the numbers 0 and 1 (meaning the variance of that feature is 0 and it will be discarded), and we expect most of the information to be represented in only a few pixels. The tradeoff to compressing the features is that we lose some detail of the original dataset.

Task 2: Implement a QNN feature map and variational ansatz

We now implement the quantum circuit building blocks of our QNN:

- 1. A **feature map** that encodes each PCA-reduced input into `n_qubits` using angle-embedding.
- 2. A **variational ansatz** with trainable parameters θ , which will be learned during training.

We will then use these building blocks inside a PennyLane QNode.

```
In [4]: # Task 2: QNN building blocks (feature map + ansatz)

# 1. Define a device with n_qubits wires in analytic mode (shots=None)
# -----YOUR CODE HERE-----
dev = qml.device("default.qubit", wires=n_qubits, shots=None)
# ---YOUR CODE ENDS HERE---

def feature_map_qnn(x, scale=1.0, entangle=True):
    """
    Simple angle-embedding feature map with optional entangling layer.

    x: 1D array-like of length n_qubits (PCA-reduced input)
    scale: rescaling factor for angles
    entangle: if True, apply a CZ ring after single-qubit rotations
    """

    # 1. Apply AngleEmbedding with rotation="Y" and the given scale
    qml.AngleEmbedding(features=x*scale, wires=range(n_qubits), rotation='Y')

    # 2. If entangle=True, apply a ring of CZ or CNOT gates
    if entangle:
        for i in range(n_qubits):
            qml.CZ(wires=[i, (i + 1) % n_qubits])

    return None

def variational_ansatz(theta):
    """
    Variational circuit with trainable parameters theta.

    theta: parameters with shape (number of layers, number of wires).

    Structure:
    - For each layer:
        - Apply RY rotations on all qubits
        - Apply a ring of CNOTs to entangle the qubits
    """

    n_layers, n_wires = theta.shape
    # -----YOUR CODE HERE-----
    for layer in range(n_layers):
        # Apply RY rotations to all qubits
        for qubit in range(n_wires):
            qml.RY(phi=theta[layer, qubit], wires=qubit)

        # Apply ring of CNOTs to entangle qubits
        for qubit in range(n_wires):
            qml.CNOT(wires=[qubit, (qubit + 1) % n_wires])
    # ---YOUR CODE ENDS HERE---
    return
```

Question 2.1 — Short answer

Explain in a few sentences how the feature map and the variational ansatz play different roles in a QNN. Relate each of them to analogous components in a classical neural network.

Answer: The feature map plays the role of transforming classical data into a quantum state using a transformation with fixed parameters and is analogous to the feature vectors that act as input into a classical neural network. The variational ansatz is a circuit that depends on **trainable parameters** and it's the depth and structure of this ansatz that control the expressivity and trainability of the QNN. The variational ansatz is analogous to the hidden layers of a classical neural network.

Question 2.2 — Reflection

Why does the absence of entangling gates in a quantum feature map or variational ansatz (or any quantum circuit in general) guarantee that the the circuit's output state can be efficiently simulated classically given a classical input such as $|0\rangle$? (2-5 sentences)

Answer: Without applying any entangling states the circuit's output state can be efficiently simulated classically because each qubit state is evolving independently. Now only single qubit states must be stored which can easily be done with classical bits.

Question 2.3 — Follow up

Given this fact, can we expect any quantum advantage from a circuit composed only of single-qubit gates in a feature map or variational ansatz? Explain why or why not.

Answer: No, we don't expect a quantum advantage because this type of circuit could be easily implemented classically. Additionally, one speedup in quantum computing stems from the entanglement of different qubits which allows for the computation of many entangled states to be executed simultaneously.

Task 3: Define the QNode and PyTorch QNN classifier

We now combine the feature map and ansatz into a QNode that returns a single expectation value, and then wrap it in a PyTorch module.

Instructions:

1. Define a QNode `qnn_circuit(x, theta)` :
 - encodes one input sample `x` using `feature_map_qnn`,
 - applies `variational_ansatz(theta)`. Try first with scale 1 and with entangling.
 - Return the expectation value of `PauliZ` on qubit 0. (Note that measuring PauliZ ensures that the expectation value is in [-1, 1]). Hint: Use the pennylane function `qml.expval`.
2. Define a `QNNClassifier(nn.Module)` that:
 - stores `theta` as an `nn.Parameter`,
 - in `forward`, loops over a mini-batch of inputs,
 - calls the QNode for each sample,
 - maps the expectation values from [-1, 1] to probabilities in [0, 1],
 - returns a tensor of shape `(batch_size, 1)`.

Question 3.1 — Short answer

Why is the expectation value of a Pauli operator (e.g. Z on one qubit) a natural scalar output for a QNN used for **binary classification**? How do we convert it into a probability in [0, 1] in a good way?

Answer: The expectation value of a Pauli operator is a natural scalar output for binary classification because it has expectation values [-1, 1] which is already a relatively common choice of labels in binary classification problems and provides symmetry around 0 for the model.

To convert the Pauli Z operator expectation values, $\langle Z \rangle = [-1, 1]$, to a probability p of [0, 1] we can apply the following equation:

$$p = \frac{1 + \langle Z \rangle}{2}.$$

```
In [5]: # Task 3: QNode and QNN classifier with PyTorch

# Decide a theta shape, e.g. for L layers and n_qubits:
# theta_shape = (n_layers, n_qubits)
# You may choose a different layout if you like.
n_layers = 2
theta_shape = (n_layers, n_qubits)

@qml.qnode(dev, interface="torch", diff_method="parameter-shift")
def qnn_circuit(x, theta):
    """
    Quantum node for a single input sample.

    x: 1D tensor with n_qubits features
    theta: trainable parameters for the ansatz
    """
    # -----YOUR CODE HERE-----
    feature_map_qnn(x, scale = 1.0, entangle = True)
    variational_ansatz(theta)
    return qml.expval(qml.PauliZ(0))
    # ---YOUR CODE ENDS HERE---

def test_qnn_circuit():
    x_sample = torch.tensor([0.1, -0.2, 0.3, 0.4], dtype=torch.float32)
    theta_sample = torch.ones(theta_shape, dtype=torch.float32)
    ev = qnn_circuit(x_sample, theta_sample)
    print("QNN circuit output (expectation value):", ev.item())
    assert np.isclose(ev.item(), -0.5613872, FP_TOL, FP_TOL)
```

```
test_qnn_circuit()

class QNNClassifier(nn.Module):
    def __init__(self, theta_shape):
        super().__init__()
        # Initialize trainable parameters theta as a PyTorch Parameter
        self.theta = nn.Parameter(torch.randn(theta_shape) * 0.01)

    def forward(self, x):
        """
        x: tensor of shape (batch_size, n_qubits)
        returns: probabilities in [0,1] of shape (batch_size, 1)
        """
        # -----YOUR CODE HERE-----
        # 1. Ensure x has a batch dimension
        if x.ndim == 1:
            x = x.unsqueeze(0)

        # 2. For each sample in the batch, call qnn_circuit(sample, self.theta)
        exp = []
        for index in range(x.shape[0]):
            sample = x[index]
            ev = qnn_circuit(sample, self.theta) # returns differentiable torch tensor
            exp.append(ev)

        # 3. Stack expectation values WITHOUT breaking autograd
        exp = torch.stack(exp) # <-- FIXED

        # 4. Map from [-1,1] to [0,1]
        p = (exp + 1.0) / 2.0

        # 5. Return p with shape (batch_size, 1)
        return p.unsqueeze(1).to(device_torch)
        # ---YOUR CODE ENDS HERE---

# Instantiate model and move to Torch device
model = QNNClassifier(theta_shape).to(device_torch)
print(model)

def test_qnn_circuit():
    test_batch1 = torch.tensor([[0.1, -0.2, 0.3, 0.4],
                                [0.5, 0.6, -0.7, 0.8]],
                                dtype=torch.float32)

    test_batch2 = torch.tensor([0.0, 0.0, 0.0, 0.0], dtype=torch.float32)

    assert model.forward(test_batch1).shape == (2, 1)
    assert model.forward(test_batch2).shape == (1, 1)

    #Assert that the output is a probability
    p = model.forward(test_batch2).item()
    assert 0.0 <= p <= 1.0

test_qnn_circuit()
```

QNN circuit output (expectation value): -0.5613872548613136
QNNClassifier()

Task 4: Training the QNN

We now train the QNN classifier on the PCA-reduced dataset.

We will use:

- `nn.BCELoss` as the loss function (binary cross-entropy on probabilities),
- `Adam` as the optimizer,
- a modest number of epochs (e.g. 15–30).

Instructions:

1. Wrap the training data in a `DataLoader`. Shuffle the data for the training loader but not the validation loader.
2. Implement the standard training loop:
 - zero gradients,
 - forward pass,
 - compute loss,
 - backpropagate,
 - optimizer step.
3. Track and print the training loss per epoch.

```
In [6]: # Create DataLoaders for training and test sets using PCA-reduced features

batch_size = 32

train_ds = TensorDataset(
    torch.tensor(Xtr_red, dtype=torch.double),
    torch.tensor(y_tr, dtype=torch.double).unsqueeze(1),
)
val_ds = TensorDataset(
    torch.tensor(Xval_red, dtype=torch.double),
    torch.tensor(y_val, dtype=torch.double).unsqueeze(1),
)
test_ds = TensorDataset(
    torch.tensor(Xte_red, dtype=torch.double),
    torch.tensor(y_te, dtype=torch.double).unsqueeze(1),
)
# -----YOUR CODE HERE-----
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
```

```
# ---YOUR CODE ENDS HERE---

print(len(train_loader), len(val_loader), len(test_loader))
assert len(train_loader) == 8
assert len(val_loader) == 2
assert len(test_loader) == 2
print(train_loader)
```

8 2 2
<torch.utils.data.dataloader.DataLoader object at 0x0000013664C046D0>

```
In [ ]: # Training Loop
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)

n_epochs = 20

history = {"loss": [], "val_loss": []}

for epoch in range(1, n_epochs + 1):
    model.train()
    epoch_loss = 0.0

    for xb, yb in train_loader:
        xb = xb.to(device_torch)
        yb = yb.to(device_torch)

        # -----YOUR CODE HERE-----
        # 1. Zero grads
        optimizer.zero_grad()
        # 2. Forward pass
        probs = model(xb)
        # 3. Compute Loss
        loss = criterion(probs, yb)
        # 4. Backpropagate
        loss.backward()
        # 5. Optimizer step
        optimizer.step()

        # ---YOUR CODE ENDS HERE---

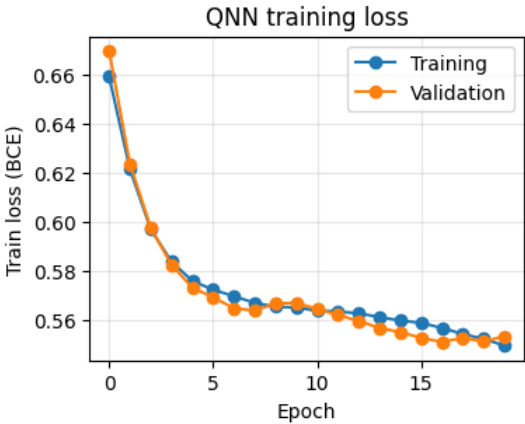
        epoch_loss += loss.item() * xb.size(0)
    epoch_loss /= len(train_loader.dataset)

    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for xb, yb in val_loader:
            xb = xb.to(device_torch)
            yb = yb.to(device_torch)
            preds = model(xb)
            loss = criterion(preds, yb)
            val_loss += loss.item() * xb.size(0)
    val_loss /= len(val_loader.dataset)

    history["loss"].append(epoch_loss)
    history["val_loss"].append(val_loss)
    print(f"Epoch {epoch:02d}/{n_epochs} - loss: {epoch_loss:.4f} - val_loss: {val_loss:.4f}")
```

```
In [ ]: # Plot training Loss

plt.figure(figsize=(4, 3))
plt.plot(history["loss"], marker='o', label="Training")
plt.plot(history["val_loss"], marker='o', label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Train loss (BCE)")
plt.title("QNN training loss")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```



Question 4.1 — Short answer

Comment briefly on the training and validation curves you obtained. Does the loss decrease steadily? Do you see signs of underfitting or overfitting?

Answer: The training loss decreases steadily but the validation loss does not. The validation loss decreases steadily until epoch 17, at which case it increases slightly to epoch 18 and flattens out in epochs 19 and 20. This is a sign of overfitting as the model is still improving its fit on the training data but flattens out on the validation data (the model is struggling to generalize).

Task 5: Evaluation on the test set

Finally, we evaluate the trained QNN on the **held-out test set**.

1. Compute predicted probabilities on the test set.
2. Convert probabilities to class predictions using a 0.5 threshold.
3. Compute and print:
 - test accuracy,
 - confusion matrix,
 - classification report (optional).

```
In [ ]: # Evaluation on test set

model.eval()
all_probs = []
all_labels = []

with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device_torch)
        yb = yb.to(device_torch)

        probs = model(xb) # (batch_size, 1)
        all_probs.append(probs.cpu())
        all_labels.append(yb.cpu())

all_probs = torch.cat(all_probs, dim=0)
all_labels = torch.cat(all_labels, dim=0)

# Convert probabilities to class predictions
y_pred = (all_probs >= 0.5).int()
y_true = all_labels.int()

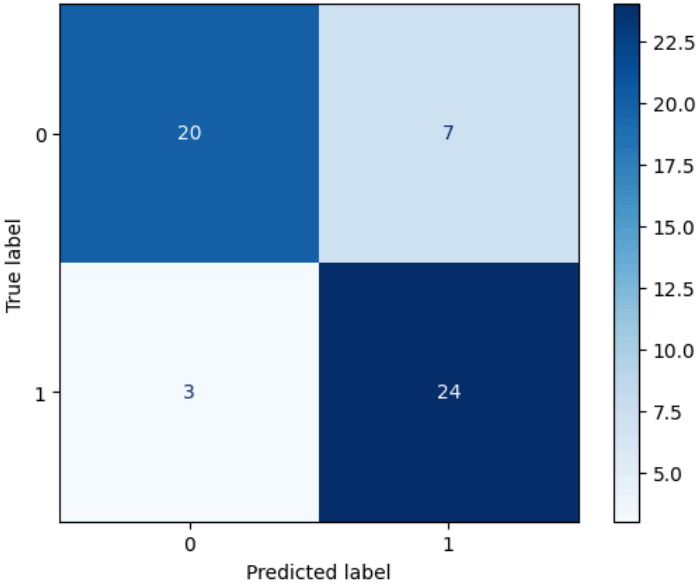
acc = accuracy_score(y_true.numpy(), y_pred.numpy())
cm = confusion_matrix(y_true.numpy(), y_pred.numpy())

print(f"Test accuracy: {acc:.3f}")
print("Confusion matrix [[TN, FP], [FN, TP]]:")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
disp.plot(cmap="Blues")
plt.show()

print(classification_report(y_true.numpy(), y_pred.numpy()))
```

Test accuracy: 0.815
Confusion matrix [[TN, FP], [FN, TP]]:
[[20 7]
 [3 24]]



	precision	recall	f1-score	support
0	0.87	0.74	0.80	27
1	0.77	0.89	0.83	27
accuracy			0.81	54
macro avg	0.82	0.81	0.81	54
weighted avg	0.82	0.81	0.81	54

Question 5.1 — Short answer

Does the confusion matrix show any interesting asymmetry (e.g., more false positives than false negatives)?

Answer: Yes, the model produced more false positives (7) than false negatives (3), indicating that the model has a tendency to predict class 1 more often than class 0.

Bonus: Gradient inspection and barren plateaus

In this bonus exercise, you will perform a more detailed gradient inspection on your QNN to connect with the concept of barren plateaus.

We will focus on three aspects:

- 1. Gradient norms at initialization for different circuit depths (`n_layers`).
- 2. Gradient norms during training (how they evolve across epochs).
- 3. Distribution of per-parameter gradients at a given point in training.

You may reuse your QNN architecture from the main tasks (feature map, ansatz, QNode, and `QNNClassifier`), or define a simplified variant dedicated to this analysis.

Bonus 0 — Helper: single-batch gradient computation

We start by writing a helper function that:

- Takes a model, a data loader, and a loss function.
- Grabs one mini-batch from the loader.
- Computes the loss and then the gradient of the loss w.r.t. `model.theta` .
- Returns the L2 norm of that gradient, divided by the square root of the number of parameters (and optionally the gradient tensor itself).

We will reuse this helper in the following subtasks.

```
In [8]: # Bonus A.0 – Helper: compute gradient norm for one batch

# Normalized gradient norm computation
def gradient_norm_for_one_batch(model, data_loader, criterion, device=device_torch):
    """
    Compute the L2 norm of the gradient of the loss w.r.t. model.theta
    using a single batch from data_loader.
    """
    model.train()
    # Take the first batch only
    xb, yb = next(iter(data_loader))
    xb = xb.to(device)
    yb = yb.to(device)

    # Zero existing gradients
    model.zero_grad()

    # Forward pass
    preds = model(xb)
    loss = criterion(preds, yb)

    # Backward pass
    loss.backward()

    # Extract gradient of theta
    grad_theta = model.theta.grad # tensor with same shape as theta
    grad_norm = torch.norm(grad_theta).item() / math.sqrt(grad_theta.numel())

    return grad_norm, grad_theta.detach().cpu(), loss.item()

# Quick sanity check (optional, only if model is already defined)
criterion = nn.BCELoss()
g_norm, g_tensor, loss_value = gradient_norm_for_one_batch(model, train_loader, criterion)
print("Gradient norm (one batch):", g_norm, "Loss:", loss_value)
```

Gradient norm (one batch): 0.035660198278154406 Loss: 0.4850851200601889

Bonus 1 — Gradient norms at initialization vs circuit depth

In this part, you will study how the gradient norm at random initialization depends on the circuit depth `n_layers` .

Instructions:

1. Choose a list of depths, e.g. `depth_list = [1, 2, 3]` (you may add more if runtime allows).
2. For each depth:
 - Rebuild a fresh QNN (same architecture as in the main task, but with that `n_layers`).
 - Randomly initialize the parameters (the default PyTorch initialization is fine).
 - Compute the gradient norm on a single training batch using `gradient_norm_for_one_batch` .
3. Store the gradient norms and print them (and optionally plot them).
4. Comment on whether deeper circuits tend to show smaller gradient norms at initialization.

You can reuse your `QNNClassifier` class, but you will need to make `n_layers` configurable.

```
In [16]: # Bonus A.1 – Gradient norm vs depth at initialization

# Make sure you have a way to construct a QNNClassifier with a given n_layers.
# For example, you can wrap the model creation in a small factory function.

def make_qnn_model(n_layers, n_qubits):
    """
    Factory for QNNClassifier with given depth n_layers and n_qubits.
    Adjust theta_shape and ansatz accordingly.
    """
    # Example: theta_shape = (n_layers, n_qubits)
    theta_shape = (n_layers, n_qubits)
    model = QNNClassifier(theta_shape).to(device_torch)
    return model

criterion = nn.BCELoss()

depth_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12] #4, 5, 6, 7] # you can add 4, 5, ... if time allows
grad_norms_depth = []

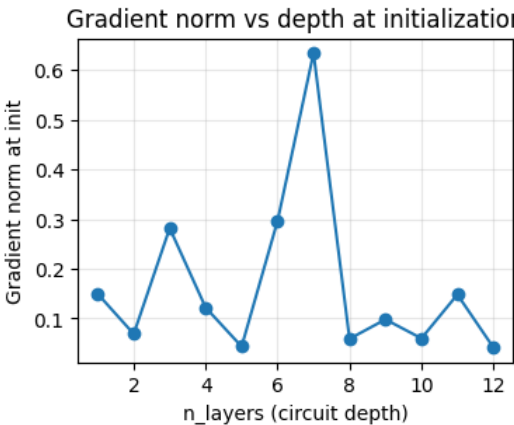
for depth in depth_list:
```



```
# -----YOUR CODE HERE-----
new_model = make_qnn_model(n_layers=depth, n_qubits=n_qubits)
g_norm, g_tensor, loss_value = gradient_norm_for_one_batch(new_model, train_loader, criterion)
grad_norms_depth.append(g_norm)
print("Gradient norm (one batch):", g_norm, "with a circuit depth of:", depth)
# ---YOUR CODE ENDS HERE---

# Optional: plot gradient norms vs depth
plt.figure(figsize=(4, 3))
plt.plot(depth_list, grad_norms_depth, marker="o")
plt.xlabel("n_layers (circuit depth)")
plt.ylabel("Gradient norm at init")
plt.title("Gradient norm vs depth at initialization")
plt.grid(alpha=0.3)
plt.show()
```

Gradient norm (one batch): 0.1491115540266037 with a circuit depth of: 1
Gradient norm (one batch): 0.06855552114714565 with a circuit depth of: 2
Gradient norm (one batch): 0.2815416452244158 with a circuit depth of: 3
Gradient norm (one batch): 0.12167588621377945 with a circuit depth of: 4
Gradient norm (one batch): 0.04314760952569015 with a circuit depth of: 5
Gradient norm (one batch): 0.2964090198247938 with a circuit depth of: 6
Gradient norm (one batch): 0.6343411179832437 with a circuit depth of: 7
Gradient norm (one batch): 0.05873399362344744 with a circuit depth of: 8
Gradient norm (one batch): 0.0978069802125295 with a circuit depth of: 9
Gradient norm (one batch): 0.059635237817257526 with a circuit depth of: 10
Gradient norm (one batch): 0.14788923075862548 with a circuit depth of: 11
Gradient norm (one batch): 0.04147619130492085 with a circuit depth of: 12



Question Bonus 1.1 — Interpretation

Comment on the gradient norms vs depth that you observed.

- Do deeper circuits tend to show smaller gradient norms at random initialization?
- How does this relate qualitatively to the idea of barren plateaus (even in this small setup)?

Answer: Based on the gradient norms shown we can see that with less than 5 layers the gradient norm is relatively small, and that the model is struggling to find a direction of decreasing gradient. We can also see that the deeper circuits tend to show smaller gradient norms, also indicative of barren plateaus, and supporting the fact that deeper networks are plagued with the problem of vanishing gradients.

Bonus 2 — Gradient norm across training epochs

We now study how the gradient norm evolves during training for a fixed circuit depth.

Instructions:

1. Pick one depth (e.g. `n_layers = 2`).
2. Train the QNN for a small number of epochs (e.g. 10) as in the main assignment.
3. At each epoch:
 - After the weight update, compute the gradient norm on one batch using `gradient_norm_for_one_batch`.
 - Store this value in a list `grad_norms_epochs`.
4. Plot gradient norm vs epoch, and compare this with the training loss curve.
5. Comment on whether gradients tend to shrink as training progresses.

Hint: You can reuse your training loop, but add a call to `gradient_norm_for_one_batch` at the end of each epoch, using the current model parameters.

```
In [25]: # Bonus A.2 – Gradient norm vs epoch for a fixed depth

# Choose a depth
depth_for_training = 2
model_train_depth_2 = make_qnn_model(depth_for_training, n_qubits)
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model_train_depth_2.parameters(), lr=0.02)

n_epochs_bonus = 10
losses_epochs = []
grad_norms_epochs = []

for epoch in range(1, n_epochs_bonus + 1):
    model_train_depth_2.train()
    epoch_loss = 0.0
    # -----YOUR CODE HERE-----
    # Training Loop for one epoch
    for xb, yb in train_loader:
        xb = xb.to(device_torch)
        yb = yb.to(device_torch)
```

```
# 1. Zero gradients
optimizer.zero_grad()

# 2. Forward pass
probs = model_train_depth_2(xb)

# 3. Compute Loss
loss = criterion(probs, yb)

# 4. Backward pass
loss.backward()

# 5. Optimizer step
optimizer.step()

# Accumulate Loss
epoch_loss += loss.item() * xb.size(0)

epoch_loss /= len(train_loader.dataset)
losses_epochs.append(epoch_loss)

# Using gradient_norm_for_one_batch
g_norm, g_tensor, loss_val = gradient_norm_for_one_batch(model_train_depth_2, train_loader, criterion, device_torch)
# ---YOUR CODE ENDS HERE---
grad_norms_epochs.append(g_norm)

print(f"Epoch {epoch:02d}/{n_epochs_bonus} - Loss: {epoch_loss:.4f}, Grad norm: {g_norm:.4e}")

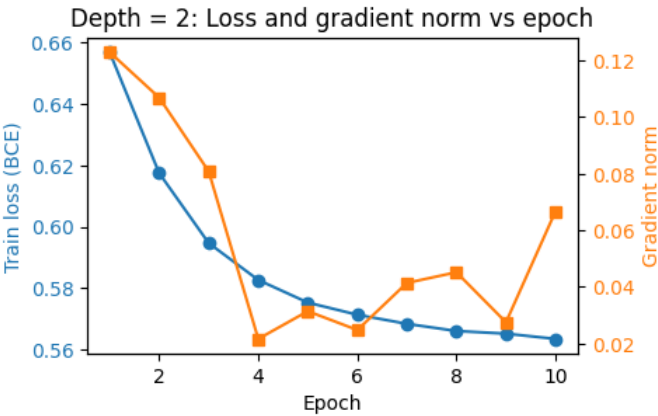
# Plot Loss and gradient norm vs epoch
fig, ax1 = plt.subplots(figsize=(5, 3))

ax1.set_xlabel("Epoch")
ax1.set_ylabel("Train loss (BCE)", color="C0")
ax1.plot(range(1, n_epochs_bonus+1), losses_epochs, marker="o", color="C0", label="Loss")
ax1.tick_params(axis="y", labelcolor="C0")

ax2 = ax1.twinx()
ax2.set_ylabel("Gradient norm", color="C1")
ax2.plot(range(1, n_epochs_bonus+1), grad_norms_epochs, marker="s", color="C1", label="Grad norm")
ax2.tick_params(axis="y", labelcolor="C1")

fig.tight_layout()
plt.title(f"Depth = {depth_for_training}: Loss and gradient norm vs epoch")
plt.show()
```

Epoch 01/10 - Loss: 0.6569, Grad norm: 1.2297e-01
Epoch 02/10 - Loss: 0.6175, Grad norm: 1.0688e-01
Epoch 03/10 - Loss: 0.5946, Grad norm: 8.0782e-02
Epoch 04/10 - Loss: 0.5826, Grad norm: 2.1575e-02
Epoch 05/10 - Loss: 0.5753, Grad norm: 3.1373e-02
Epoch 06/10 - Loss: 0.5713, Grad norm: 2.4714e-02
Epoch 07/10 - Loss: 0.5683, Grad norm: 4.1362e-02
Epoch 08/10 - Loss: 0.5660, Grad norm: 4.5132e-02
Epoch 09/10 - Loss: 0.5651, Grad norm: 2.7461e-02
Epoch 10/10 - Loss: 0.5634, Grad norm: 6.6695e-02



```
In [21]: # Bonus A.2 – Gradient norm vs epoch for a fixed depth

# Choose a depth
depth_for_training = 7
model_train = make_qnn_model(depth_for_training, n_qubits)
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model_train.parameters(), lr=0.02)

n_epochs_bonus = 10
losses_epochs = []
grad_norms_epochs = []

for epoch in range(1, n_epochs_bonus + 1):
    model_train.train()
    epoch_loss = 0.0
    # -----YOUR CODE HERE-----
    # Training Loop for one epoch
    for xb, yb in train_loader:
        xb = xb.to(device_torch)
        yb = yb.to(device_torch)

        # 1. Zero gradients
        optimizer.zero_grad()

        # 2. Forward pass
        probs = model_train(xb)

        # 3. Compute Loss
        loss = criterion(probs, yb)
```

```
# 4. Backward pass
loss.backward()

# 5. Optimizer step
optimizer.step()

# Accumulate Loss
epoch_loss += loss.item() * xb.size(0)

epoch_loss /= len(train_loader.dataset)
losses_epochs.append(epoch_loss)

# Using gradient_norm_for_one_batch
g_norm, g_tensor, loss_val = gradient_norm_for_one_batch(model_train, train_loader, criterion, device_torch)
# ---YOUR CODE ENDS HERE---
grad_norms_epochs.append(g_norm)

print(f"Epoch {epoch:02d}/{n_epochs_bonus} - Loss: {epoch_loss:.4f}, Grad norm: {g_norm:.4e}")

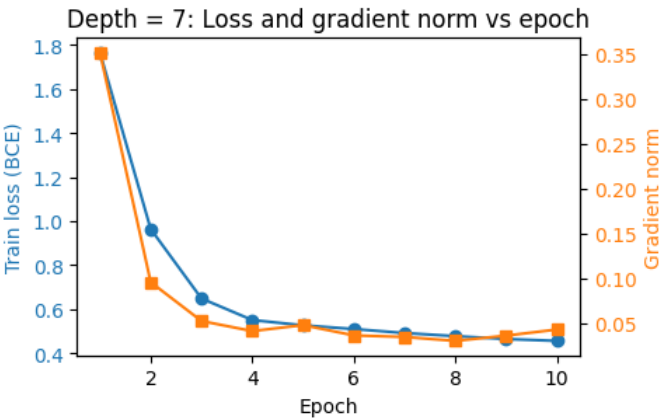
# Plot Loss and gradient norm vs epoch
fig, ax1 = plt.subplots(figsize=(5, 3))

ax1.set_xlabel("Epoch")
ax1.set_ylabel("Train loss (BCE)", color="C0")
ax1.plot(range(1, n_epochs_bonus+1), losses_epochs, marker="o", color="C0", label="Loss")
ax1.tick_params(axis="y", labelcolor="C0")

ax2 = ax1.twinx()
ax2.set_ylabel("Gradient norm", color="C1")
ax2.plot(range(1, n_epochs_bonus+1), grad_norms_epochs, marker="s", color="C1", label="Grad norm")
ax2.tick_params(axis="y", labelcolor="C1")

fig.tight_layout()
plt.title(f"Depth = {depth_for_training}: Loss and gradient norm vs epoch")
plt.show()
```

Epoch 01/10 - Loss: 1.7666, Grad norm: 3.5156e-01
Epoch 02/10 - Loss: 0.9630, Grad norm: 9.5535e-02
Epoch 03/10 - Loss: 0.6474, Grad norm: 5.2287e-02
Epoch 04/10 - Loss: 0.5494, Grad norm: 4.1090e-02
Epoch 05/10 - Loss: 0.5255, Grad norm: 4.7715e-02
Epoch 06/10 - Loss: 0.5086, Grad norm: 3.6258e-02
Epoch 07/10 - Loss: 0.4904, Grad norm: 3.4503e-02
Epoch 08/10 - Loss: 0.4762, Grad norm: 3.0162e-02
Epoch 09/10 - Loss: 0.4632, Grad norm: 3.6032e-02
Epoch 10/10 - Loss: 0.4547, Grad norm: 4.2822e-02



Question Bonus 2.1 — Observation

Describe how the gradient norm changes during training: Does it decrease, stay roughly constant, or fluctuate?

Answer: We calculated the gradient norm for 10 training epochs for a network depth of both 2 and 7.

- **Depth of 2:** From Bonus Task 1, we expect that the model will fall into a barren plateau at a depth of 2 which will result in a network that does not fit well to the data. This is reflected in the plot for a depth of 2 where we see the gradient norm fluctuating and generally not decreasing in the expected manner.
- **Depth of 7:** From Bonus Task 1, we expect the model will be able to learn better and detect non-zero gradient norms. When we calculated the gradient norm for a network depth of 7 we saw that the gradient norm decreased steadily, aligning well with the training loss.

Question Bonus 2.2 — Interpretation

How does this relate to the changes in the training loss?

Answer: With a depth of 2 we saw that the gradient norm did not relate strongly to the training loss and instead fluctuated both above and below the training loss. With a depth of 7 we observed that the gradient norm behaved similarly to the training loss, decreasing with each epoch.

Question Bonus 2.3 — Interpretation

In general, if our QNN suffered from a barren plateau, how would that affect the gradient norm?

Answer: If our QNN suffers from a barren plateau we would expect the gradient norm to be near 0 and not decreasing with each epoch, instead it would be fluctuating around near 0 values from epoch to epoch.

Bonus 3 — Distribution of per-parameter gradients

So far, we have focused on the normalized L2 norm of the gradient.
Now, we take a closer look at the distribution of individual gradient components.

Instructions:

- 1. Using a model (for some depth), compute the gradient of the loss w.r.t. `model.theta` on one batch.
- 2. Extract all gradient entries as a 1D array and plot a histogram of their values.

- Do step 1 and 2:
- once at initialization, and
 - once after training, and compare the two histograms.

This gives a sense of whether gradients are broadly distributed or tend to concentrate tightly around zero.

```
In [26]: # Bonus A.3 – Gradient distribution (histogram)

def gradient_histogram(model, data_loader, criterion, device=device_torch, title=""):
    """
    Compute gradients on one batch and plot a histogram of gradient values.
    """
    model.train()
    xb, yb = next(iter(data_loader))
    xb = xb.to(device)
    yb = yb.to(device)

    # -----YOUR CODE HERE-----

    # Zero existing gradients
    model.zero_grad()

    # Forward pass
    preds = model(xb)
    loss = criterion(preds, yb)

    # Backward pass
    loss.backward()
    # ---YOUR CODE ENDS HERE---

    grad_vals = model.theta.grad.detach().cpu().numpy().ravel()

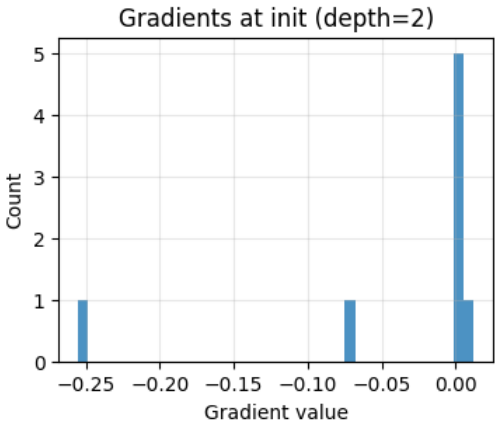
    plt.figure(figsize=(4, 3))
    plt.hist(grad_vals, bins=40, alpha=0.8)
    plt.xlabel("Gradient value")
    plt.ylabel("Count")
    plt.title(title)
    plt.grid(alpha=0.3)
    plt.show()

    print(f"{title} – mean grad: {grad_vals.mean():.2e}, std grad: {grad_vals.std():.2e}")
    return grad_vals

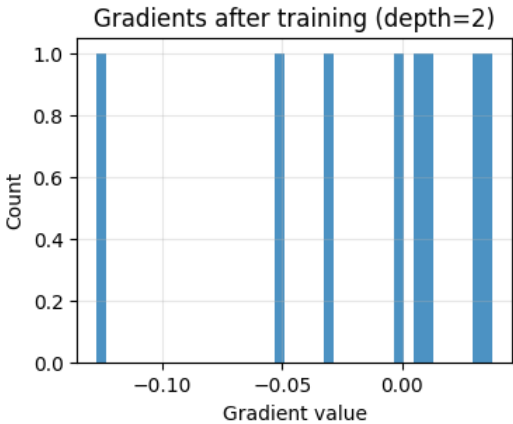
criterion = nn.BCELoss()
depth = 2
# Example: histogram at initialization for specific depth
model_init = make_qnn_model(depth, n_qubits)
print("Gradient distribution at initialization (depth=2):")
_ = gradient_histogram(model_init, train_loader, criterion, device=device_torch,
                      title="Gradients at init (depth=2)")

# Example: histogram after training for depth=2 (reuse model_train from Bonus A.2 if available)
try:
    print("Gradient distribution after training (depth=2):")
    _ = gradient_histogram(model_train_depth_2, train_loader, criterion, device=device_torch,
                          title="Gradients after training (depth=2)")
except NameError:
    print("model_train not found. Run Bonus A.2 first or train a model for this analysis.")
```

Gradient distribution at initialization (depth=2):



Gradients at init (depth=2) – mean grad: -3.83e-02, std grad: 8.56e-02
Gradient distribution after training (depth=2):



Gradients after training (depth=2) – mean grad: -1.55e-02, std grad: 5.05e-02

Compare the gradient histograms you obtained:

```
In [24]: # Bonus A.3 – Gradient distribution (histogram)

def gradient_histogram(model, data_loader, criterion, device=device_torch, title=""):
    """
    Compute gradients on one batch and plot a histogram of gradient values.
    """
    model.train()
    xb, yb = next(iter(data_loader))
    xb = xb.to(device)
    yb = yb.to(device)

    # -----YOUR CODE HERE-----

    # Zero existing gradients
    model.zero_grad()

    # Forward pass
    preds = model(xb)
    loss = criterion(preds, yb)

    # Backward pass
    loss.backward()
    # ---YOUR CODE ENDS HERE---

    grad_vals = model.theta.grad.detach().cpu().numpy().ravel()

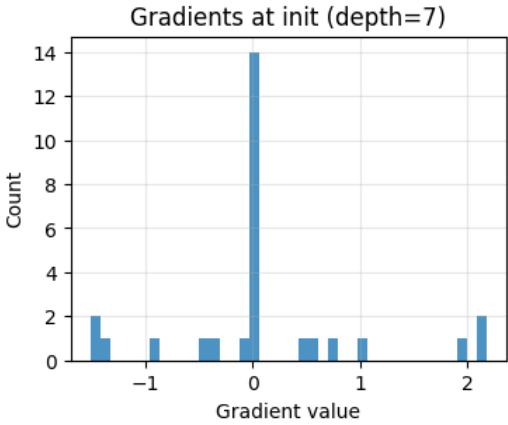
    plt.figure(figsize=(4, 3))
    plt.hist(grad_vals, bins=40, alpha=0.8)
    plt.xlabel("Gradient value")
    plt.ylabel("Count")
    plt.title(title)
    plt.grid(alpha=0.3)
    plt.show()

    print(f"{title} – mean grad: {grad_vals.mean():.2e}, std grad: {grad_vals.std():.2e}")
    return grad_vals

criterion = nn.BCELoss()
depth = 7
# Example: histogram at initialization for specific depth
model_init = make_qnn_model(depth, n_qubits)
print("Gradient distribution at initialization (depth=7):")
_ = gradient_histogram(model_init, train_loader, criterion, device=device_torch,
                      title="Gradients at init (depth=7)")

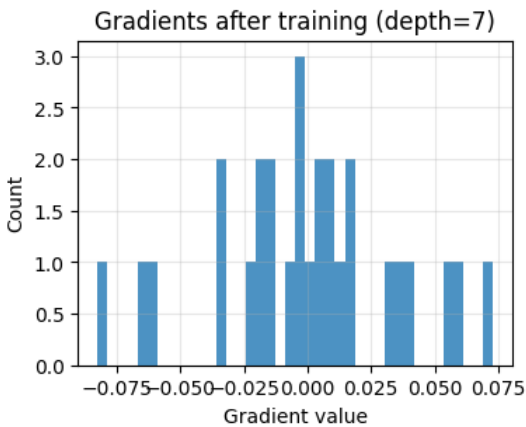
# Example: histogram after training for depth=2 (reuse model_train from Bonus A.2 if available)
try:
    print("Gradient distribution after training (depth=7):")
    _ = gradient_histogram(model_train, train_loader, criterion, device=device_torch,
                        title="Gradients after training (depth=7)")
except NameError:
    print("model_train not found. Run Bonus A.2 first or train a model for this analysis.")
```

Gradient distribution at initialization (depth=7):



Gradients at init (depth=7) – mean grad: 1.10e-01, std grad: 8.94e-01

Gradient distribution after training (depth=7):



Gradients after training (depth=7) – mean grad: -4.65e-04, std grad: 3.55e-02

Question Bonus 3.1 — Observation

Are gradients broadly distributed or sharply concentrated near zero?

Answer:

- **Depth of 2:** Before training the gradients are sharply concentrated near zero, with very little variance (ranging from -0.25 to 0).
- **Depth of 7:** Before training there is a sharp concentration of gradients at exactly 0, but there is a much larger variance than for a depth of 2, with gradient values ranging from roughly -1.5 to 2.

Question Bonus 3.2 — Short answer

Does training make them more or less concentrated?

Answer: Training makes the gradients more broadly distributed and less concentrated at zero.

- **Depth of 2:** Training makes the gradients less broadly distributed by a small amount, with gradient values ranging from roughly -0.13 to 0.3.
- **Depth of 7:** Training makes the gradients much less broadly distributed, with gradient values ranging from roughly -0.075 to 0.075. This large decrease in variance and larger concentration around zero indicates that the model was able to train effectively.

Question Bonus 3.3 — Reflection

How would a highly concentrated distribution of very small gradients relate to barren plateaus?

Answer: A highly concentrated distribution of very small gradients indicates a barren plateau.

Feedback to us

Optional question

Was there any part of the tasks where you struggled for some "unnecessary" reason? (Errors in the notebook, bad instructions etc.)

Answer: [Your optional answer here]

Disclosure of AI Usage (Mandatory)

Fill in this part disclosing any AI usage before submitting the assignment by describing your use of LLMs or other AI-based tools in this assignment.

For each task, we ask you to provide information about:

- **Tools/models used.**
- **Per-task usage:** for each task, a brief summary of what the tool was used for.
- **Prompts/transcripts:** main prompts or a summary of interactions (a link or screenshot is acceptable if long).
- **Validation:** how you checked and verified the correctness of AI-generated outputs (tests run, docs consulted, comparisons, plots etc.).

Disclosure:

- **Task 0:** None.
- **Task 1:** None.
- **Task 2:** None.
- **Task 3:** The way we were storing the values was breaking autograd whenever we called `loss.backward()` and giving the warning `"RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn"`. Used Chat-GPT to tell us the fix, which was to use `torch.stack()`. Also used Chat-GPT to resolve the issue of torch tensors being on different devices, and used the fix of `.to(device_torch)`.

- **Task 4:** None.
- **Task 5:** None.
- **Bonus 1:** None.
- **Bonus 2:** None.
- **Bonus 3:** None.

If you did not use any AI tools for a given exercise, specify this by writing "None". If you did not complete the bonus exercises, you can leave those fields empty.